Resumen de estudio - Semana 4
CIB12 - Aprendizaje de Maquina y Mineria de Datos

1. 1. Tema

Big Data Offline Batch Processing (Cap. 2: Marco técnico de procesamiento offline). En esta semana se estudia el *framework técnico* que soporta el procesamiento por lotes offline, centrado en: **HDFS** (almacenamiento), **Hive** (data warehouse), **Spark/SparkSQL** (análisis batch) y **herramientas de recolección/carga** (Sqoop y Loader).

Idea central

El procesamiento offline por lotes se apoya en un **ecosistema modular**: **almacenar** grandes volúmenes (HDFS), **organizar** y **consultar** como almacén analítico (Hive), **procesar** eficientemente en batch (Spark/SparkSQL) y **mover/cargar** datos desde/hacia sistemas externos (Sqoop/Loader).

2. 2. Resumen de la semana

- **HDFS**: sistema distribuido inspirado en GFS, con **tolerancia a fallos**, **alto throughput** y **soporte para archivos grandes** (TB/PB); no es ideal para muchos archivos pequeños, escritura aleatoria o lecturas de baja latencia.
- **Hive**: herramienta de **data warehouse** sobre Hadoop para consulta/gestión distribuida (PB); soporta ETL, múltiples motores (MapReduce/Tez/Spark), acceso a HDFS y HBase; organiza datos con tablas internas/externas, particiones y buckets.
- **Spark/SparkSQL**: motor batch **en memoria** (mayor eficiencia que MapReduce), con conceptos clave (RDD, shuffle, dependencias, stages, transformations/actions) y un módulo SQL (SparkSQL) orientado a **datos estructurados** en escenarios **no en tiempo real**.
- **Sqoop/Loader**: herramientas para **transferir/cargar** datos entre Hadoop (HDFS/Hive/HBase) y bases relacionales u otros sistemas; Sqoop explica import/export con paralelismo vía *maps*; Loader agrega interfaz gráfica, seguridad y alta disponibilidad.

3. 3. Conceptos clave

HDFS
Sistema de archivos distribuido para **almacenamiento masivo** y acceso tipo **streaming**.

NameNode / DataNode
NameNode gestiona **metadatos**; DataNodes almacenan **bloques** y replican datos.

Hive
Data warehouse en Hadoop; abstrae datos con **tablas** y consultas tipo SQL (HQL).

Tabla interna vs externa
Define si al `DROP` se elimina **datos+metadatos** (interna) o **solo metadatos** (externa).

UDF/UDAF/UDTF
Funciones definidas por el usuario: fila→fila, múltiples filas→una fila, fila→múltiples filas.

Data mart vs Data warehouse
Data mart: orientado a **departamentos** y cubos; Data warehouse: **capas/modelos** para múltiples análisis.

Spark
Motor distribuido **en memoria** para batch, analítica y minería de datos.

RDD
Estructura distribuida **inmutable** y **particionable**; base del procesamiento Spark.

Shuffle
Reorganización de datos que impacta rendimiento y divide el DAG en **stages**.

SparkSQL
Módulo SQL para datos estructurados; compila SQL a planes ejecutables sobre Spark.

Sqoop
Transferencia de datos entre Hadoop y BD relacionales (import/export) con paralelismo.

Loader
Herramienta de carga (basada/optimizada sobre Sqoop) con interfaz gráfica, seguridad y HA.

4. 4. Desarrollo del tema

4.1. 4.1 HDFS (Data Storage)

Diseño y propósito. HDFS se basa en el enfoque de sistemas distribuidos tipo GFS, y prioriza:

- **Alta tolerancia a fallos**: se asume que el hardware puede fallar.
- **Alto throughput**: ideal para accesos masivos a datos.
- **Archivos grandes**: almacenamiento a escala TB/PB.

Adecuado / No adecuado.

- **Adecuado**: almacenamiento y acceso de **archivos grandes**; acceso **streaming**.
- **No adecuado**: **muchos archivos pequeños**; **escritura aleatoria**; **lectura de baja latencia**.

Arquitectura básica (idea del diagrama). Cliente↔NameNode para operaciones de **metadatos**; Cliente↔DataNodes para **operaciones de bloques**; **replicación** de bloques y organización por **racks**.

Comandos frecuentes (`hdfs dfs`).

- `-cat`: muestra contenido de archivo.
- `-ls`: lista directorios/archivos.
- `-rm`: elimina archivo (pasa por papelera si está activa).
- `-put` / `-get`: subir/bajar datos hacia/desde HDFS.
- `-mkdir`: crea directorio.
- `-chmod` / `-chown`: permisos/propietario.

Administración: `hdfs dfsadmin -safemode` (modo seguro), `-report` (estado).

Papelera (Trash). Al borrar, HDFS puede mover archivos a papelera. Se configura por tiempo (minutos) con `fs.trash.interval` en `core-site.xml` (ej. 1440 min).

4.2. 4.2 Hive (Data Warehouse)

Definición. Hive es una herramienta de data warehouse sobre Hadoop que soporta **consulta y gestión distribuida** a escala PB.

Características principales.

- ETL **flexible y conveniente**.
- Soporta motores: **MapReduce**, **Tez**, **Spark**.
- Acceso directo a **HDFS** y **HBase**.
- Fácil de usar y programar (enfoque SQL/HQL).

Arquitectura (según diagrama). Interfaces (CLI/Web/JDBC/ODBC) → **Driver** (Compiler/Optimizer/Executor) → **MetaStore** (metadatos) y ejecución en el ecosistema Hadoop.

Modelo de almacenamiento. Database → tablas (internas/externas) → **particiones** → **buckets**. Se considera además **skewed data** (datos sesgados) vs **normal data**.

Tablas internas vs externas (comparación)

Acción	Interna	Externa
`CREATE/LOAD`	Mueve datos al directorio del repositorio	No mueve la ubicación física
`DROP`	Borra metadatos y datos	Borra solo metadatos
Conversión	`alter table tableName set tblproperties('EXTERNAL'='TRUE/FALSE');`	

Funciones en Hive.

- Ver funciones: `show functions;`
- Ver ayuda: `desc function upper;` / `desc function extended upper;`
- Tipos comunes: matemáticas (`round, abs, rand`), fechas (`to_date, current_date`), cadenas (`trim, length, substr`).

UDF en Hive (tipos).

- **UDF**: recibe **1 fila** y devuelve **1 fila**.
- **UDAF**: recibe **múltiples filas** y devuelve **1 fila**.
- **UDTF**: recibe **1 fila** y devuelve **múltiples filas**.

Procedimiento de desarrollo UDF (resumen).

1. Heredar de `org.apache.hadoop.hive.ql.exec.UDF`.
2. Implementar `evaluate()` con la lógica.
3. Comprimir y subir a HDFS.
4. Crear función temporal en Hive.
5. Invocar la función en consultas.

Optimización en Hive (ideas clave).

- **Data skew**: exceso de datos concentrados en pocos servidores; ralentiza todo el cómputo. Causas típicas: `group by`, `distinct count`, `join`.
- Agregación en `map` y balanceo: `set hive.map.aggr=true;` y `set hive.groupby.skewindata=true;`
- **Map-side join**: tabla pequeña en memoria al unir con tabla grande: `set hive.auto.convert.join=true;`
- **Ejecución paralela**: fases del plan en paralelo: `set hive.exec.parallel=true;` y `set hive.exec.parallel.thread.number=8;`

Data mart vs Data warehouse (diferencia conceptual)

- **Data mart**: orientado a **departamentos/usuarios específicos**; almacena por dimensiones/métricas/jerarquías para generar **cubos** y apoyar decisiones.
- **Data warehouse**: organiza datos por **capas** y **modelos** (modular) para cubrir múltiples requerimientos analíticos.

Capas típicas en warehouse con Hive.

- **ODS**: capa de datos crudos.
- **DWD**: misma estructura/granularidad que el origen; facilita limpieza.
- **DWS**: agregación ligera a partir de DWD.
- **ADS**: datos para reportes/estadísticas.

Ventajas de la estratificación (layering).

- Simplifica problemas complejos dividiendo en pasos.
- Reduce R&D repetido: estandariza y reutiliza resultados intermedios.
- Aísla datos crudos de datos estadísticos: reduce excepciones/sensibilidad.

4.3. 4.3 Spark y SparkSQL (Offline Analysis)

Spark (visión general). Motor batch distribuido **basado en memoria**; divide tareas y las ejecuta en múltiples CPUs. Al guardar intermedios en memoria reduce I/O a disco y aumenta velocidad; es útil en procesamiento y minería de datos.

Escenarios de aplicación de Spark.

- Procesamiento de datos (rápido, tolerante a fallos, escalable).
- Cómputo iterativo (lógicas complejas).
- Minería/análisis con algoritmos de ML y data mining.
- Procesamiento de streams con latencia a nivel de segundos.
- Análisis con SQL/DSL sobre datos estructurados.

Spark vs MapReduce (comparación).

- Mayor rendimiento (se reporta mejora grande al usar memoria).
- Intermedios en memoria ⇒ eficiente en iteraciones y menor latencia en batch.
- Modelo más flexible y mayor eficiencia de desarrollo.
- Mayor tolerancia a fallos (lineage).

RDD. Estructura distribuida, **solo lectura** y **particionable**. Se crea desde HDFS/almacenamientos compatibles, desde un RDD padre, o desde colecciones. Se puede cachear con distintos niveles; por defecto en memoria y desborda a disco si falta memoria.

Shuffle, dependencias y stages.

- **Shuffle** afecta la velocidad y separa el DAG en **stages**.
- **Narrow dependency**: una partición del padre alimenta a una partición del hijo (p. ej., `map`, `filter`, `union`).
- **Wide dependency**: una partición del padre puede alimentar varias del hijo (p. ej., `groupByKey`, `join` no coparticionado); suele implicar shuffle.

Transformations vs Actions.

- **Transformation**: operación **perezosa** que crea un nuevo RDD (no ejecuta aún).
- **Action**: operación que **dispara** el cálculo; devuelve resultado o escribe en almacenamiento.

Objetos clave.

- **SparkConf**: configuración (pares clave-valor). Prioridad: **código** > **parámetros dinámicos** > **archivo de config**.
- **SparkContext**: entrada y conexión al clúster; crea RDDs y registra info de ejecución.
- **SparkSession**: (Spark 2.0) entrada unificada; encapsula SparkConf/SparkContext y APIs.

SparkSQL (definición). Módulo para procesar **datos estructurados** con SQL. Convierte SQL en planes de ejecución (sobre RDD) y los ejecuta en SparkCore.

Uso y limitación importante.

- Usando **SparkSession**: tareas distribuidas como app Spark.
- Usando **JDBC**: el driver envía a **JDBCServer** del clúster; es **punto único** y puede ser cuello de botella (no ideal para mucho volumen y alta concurrencia).

DataSet (concepto en SparkSQL).

- Conjunto **fuertemente tipado** de objetos; transformable en paralelo (funcional/relacional).
- Representado por **Catalyst logical plan**; datos en binario para operar (sort/filter/shuffle) sin deserializar.
- **Lazy**: calcula solo cuando se ejecuta un *action*; optimiza planes lógicos y genera plan físico eficiente/distribuido.

Escenario de aplicación de SparkSQL.

- **Aplicable**: datos estructurados; escenarios **sin requerimiento de tiempo real**; datos a escala PB.
- **No aplicable**: **consulta en tiempo real**.

Ejemplos de consulta simple (DataFrame).

- `df.select(ïd","name").show()`
- `df.select($ïd",$"name").where($"name`== "bbb").show()`
- `df.select($ïd",$"name").orderBy($"name".desc).show`
- `df.select($ïd",$"name").sort($"name".desc).show`

4.4. 4.4 Herramientas de recolección/carga: Sqoop y Loader

Motivación. La variedad de fuentes de datos hace desafiante la recolección y la carga; se presentan herramientas comunes: **Sqoop** y **Loader**.

Sqoop (definición e idea de uso).

- Proyecto iniciado en 2009; de módulo tercero a proyecto Apache independiente.
- Transfiere datos entre Hadoop (HDFS/Hive) y BD tradicionales (MySQL, Oracle, PostgreSQL, etc.) mediante **import/export**.

Principio de importación (Sqoop).

- Se debe indicar `split-by`. Sqoop divide el rango de valores y asigna regiones a distintos **maps**.
- Cada **map** procesa filas de la BD y escribe a HDFS.
- El método de partición depende del tipo del campo; para `int`, se usan máximo/mínimo y `num-mappers` para determinar regiones.

Principio de exportación (Sqoop).

- Obtiene esquema/metainformación de la tabla destino y hace **matching** de campos.
- Importación paralela: archivos en Hadoop se dividen en **shards**; cada shard lo importa un **map task**.

Loader (definición). Herramienta de carga para intercambiar datos/archivos entre MRS y BD relacionales o entre sistemas de archivos; ofrece interfaz asistida (wizard), programación periódica y configuración de múltiples fuentes, limpieza/conversión y almacenamiento del clúster. Se apoya en Sqoop con optimizaciones y extensiones.

Escenarios (Loader). Conecta Hadoop (HDFS/HBase/Hive) con **RDB**, **SFTP** y fuentes personalizadas.

Características (Loader).

- **Gráfico**: interfaz de configuración y monitoreo.
- **Confiable**: servidores en activo/standby; jobs vía MapReduce; reintentos; sin datos residuales tras fallo.
- **Alto rendimiento**: MapReduce para procesamiento paralelo.
- **Seguro**: autenticación Kerberos y gestión de permisos de jobs.

Errores comunes o confusiones frecuentes

- **Confundir SparkSQL con "tiempo real"**: SparkSQL está orientado a análisis batch/estructurado y se indica como **no aplicable** para consulta en tiempo real.
- **Elegir mal el tipo de tabla en Hive**: en tablas **internas**, `DROP` borra **datos+metadatos**; en **externas** se borra **solo metadatos**.
- **Ignorar data skew**: `group by`, `distinct` y `join` pueden concentrar carga y ralentizar todo el job si no se mitiga.
- **Pensar que HDFS sirve para todo**: no es ideal para **muchos archivos pequeños** ni para **lectura de baja latencia**.

5. 5. Procesos, arquitectura o flujo del tema

5.1. 5.1 Flujo de almacenamiento y procesamiento (visión integrada)

1. **Ingreso/carga**: datos llegan desde BD relacionales o sistemas externos (Sqoop/Loader) hacia HDFS/Hive/HBase.
2. **Almacenamiento base**: HDFS guarda datos masivos (bloques replicados); NameNode coordina metadatos.
3. **Organización analítica**: Hive define esquemas/tablas/particiones/buckets y administra metadatos (MetaStore).
4. **Cómputo batch**: Spark procesa con RDD (transformations → actions), minimizando I/O por uso de memoria; SparkSQL traduce SQL a planes ejecutables.
5. **Optimización**: en Hive se ajustan parámetros (agregación en map, skew, joins, paralelismo) según patrón de carga.

5.2. 5.2 Flujos clave explicados en texto

- **Sqoop import**: `split-by` define partición del rango → múltiples **maps** leen porciones → escriben a HDFS en paralelo.
- **Sqoop export**: identifica esquema destino → divide archivos en shards → múltiples **maps** exportan en paralelo.
- **Spark ejecución**: transformations construyen DAG (lazy) → shuffle separa stages → actions disparan ejecución distribuida.
- **Hive consulta**: interfaz (CLI/JDBC) → Driver compila/optimiza → ejecuta en motor (MapReduce/Tez/Spark) con metadatos en MetaStore.

6. 6. Herramientas, algoritmos o componentes importantes

6.1. 6.1 Componentes (y su rol)

Componente	Rol principal
HDFS	Almacenamiento distribuido masivo; bloques replicados; alto throughput.
Hive	Data warehouse en Hadoop; ETL y consultas tipo SQL (HQL); MetaStore.
Spark	Procesamiento batch en memoria; RDD; stages/shuffle; fault-tolerance por lineage.
SparkSQL	Consultas SQL sobre Spark; planes optimizados (Catalyst) y ejecución distribuida.
Sqoop	Import/export entre BD relacionales y Hadoop (HDFS/Hive/HBase).
Loader	Carga con interfaz gráfica, HA, seguridad Kerberos y jobs MapReduce.

6.2. 6.2 Parámetros y comandos que suelen preguntar

- Hive (skew y agregación): `hive.map.aggr`, `hive.groupby.skewindata`
- Hive (joins): `hive.auto.convert.join`
- Hive (paralelo): `hive.exec.parallel`, `hive.exec.parallel.thread.number`
- HDFS (papelera): `fs.trash.interval` en `core-site.xml`

7. 7. Aplicaciones o ejemplos

7.1. 7.1 Ejemplos típicos (sin código)

- **Carga desde BD**: importar tablas desde MySQL/Oracle a HDFS para análisis batch (Sqoop).
- **Construcción de warehouse**: definir capas ODS/DWD/DWS/ADS en Hive para estandarizar limpieza, agregación y reporting.
- **Analítica estructurada**: ejecutar consultas con SparkSQL sobre datos estructurados (sin requerimiento de tiempo real).
- **Orquestación operativa**: usar Loader para trabajos periódicos con configuración asistida, reintentos y monitoreo.

8. 8. Resumen para memorizar

- **HDFS** = almacenamiento masivo (TB/PB), throughput alto, tolerante a fallos; no para muchos archivos pequeños ni baja latencia.
- **Hive** = warehouse sobre Hadoop (PB), ETL + SQL/HQL; tablas internas vs externas; particiones/buckets; UDF/UDAF/UDTF.
- **Optimización Hive** = manejar **data skew**, map-side join y paralelismo.
- **Spark** = batch en memoria con RDD; shuffle divide stages; transformations (lazy) vs actions (ejecutan).
- **SparkSQL** = SQL para datos estructurados; útil en batch, no para tiempo real; JDBCServer puede ser cuello de botella.
- **Sqoop** = import/export paralelo con maps (`split-by` y `num-mappers`).
- **Loader** = carga "enterprise": interfaz gráfica, HA, Kerberos, jobs MapReduce con reintentos.

9. 9. Preguntas probables de cuestionario

1. **¿Para qué está diseñado HDFS?**
   R: Para almacenamiento distribuido masivo (archivos grandes) con alta tolerancia a fallos y alto throughput.
2. **Menciona 3 casos en los que HDFS no es adecuado.**
   R: Muchos archivos pequeños, escritura aleatoria, lectura de baja latencia.
3. **¿Qué roles cumplen NameNode y DataNode?**
   R: NameNode gestiona metadatos; DataNode almacena bloques y replica datos.
4. **¿Qué es Hive y qué escala soporta?**
   R: Un data warehouse sobre Hadoop para consulta/gestión distribuida a escala PB.
5. **Diferencia clave entre tabla interna y externa en Hive al hacer `DROP`.**
   R: Interna: borra metadatos y datos; Externa: borra solo metadatos.
6. **¿Qué significan UDF, UDAF y UDTF en Hive?**
   R: UDF: fila→fila; UDAF: múltiples filas→una fila; UDTF: fila→múltiples filas.
7. **¿Qué es data skew en Hive y qué operaciones lo suelen causar?**
   R: Concentración de datos en pocos nodos; típico en `group by`, `distinct` y `join`.
8. **¿Para qué sirve `hive.auto.convert.join=true`?**
   R: Habilita *map-side join* (tabla pequeña en memoria al unir con una grande).
9. **¿Qué es un RDD en Spark?**
   R: Dataset distribuido resiliente, inmutable y particionable; base del procesamiento.
10. **Diferencia entre *transformation* y *action* en Spark.**
    R: Transformation es lazy y crea RDD; Action dispara ejecución y devuelve/escribe resultados.
11. **¿Qué limitación se menciona al usar SparkSQL vía JDBCServer?**
    R: Es un punto único y puede volverse cuello de botella con alto volumen y concurrencia.
12. **Explica el principio de importación en Sqoop en una frase.**
    R: Divide el rango por `split-by` y paraleliza la lectura con múltiples **maps** que escriben a HDFS.
13. **Menciona 2 características de Loader.**
    R: Interfaz gráfica de configuración/monitoreo; alta disponibilidad (active/standby) con reintentos; seguridad Kerberos (cualquiera 2).

10. 10. Mini glosario

Throughput
Capacidad de transferir/procesar grandes volúmenes de datos por unidad de tiempo.

Latencia
Tiempo de respuesta percibido en lectura/consulta; HDFS no prioriza baja latencia.

Metadatos
Información sobre datos (rutas, bloques, réplicas); central en NameNode y MetaStore.

Partición (Hive)
División lógica de una tabla por valores de una columna para mejorar gestión/consulta.

Bucket (Hive)
División física/lógica adicional que distribuye datos en archivos para optimizar joins/agrupaciones.

Skew (sesgo)
Desbalance de distribución de datos que genera cuellos de botella en cómputo.

DAG/Stage
Grafo de ejecución en Spark; un stage agrupa tareas hasta que un shuffle obliga a separar etapas.

Catalyst
Plan lógico/optimizador asociado a SparkSQL (se usa para representar/optimizar consultas).

Split-by (Sqoop)
Campo usado para particionar rangos y distribuir trabajo de importación entre maps.

Kerberos
Mecanismo de autenticación usado para seguridad en entornos Hadoop (mencionado en Loader).